

C Programming Assignment Solutions

Question 1: Variable Declaration and Address Printing

For the given structure, declare the variable type, and print their values as well as addresses.

Answer:

```
#include <stdio.h>

int main() {
    int La = 345;
    float Fb = 4.5;
    char Chvar = 'Z';

    printf("La: Value = %d, Address = %p\n", La, &La);
    printf("Fb: Value = %.1f, Address = %p\n", Fb, &Fb);
    printf("Chvar: Value = %c, Address = %p\n", Chvar, &Chvar);

    return 0;
}
```

Output:

```
La: Value = 345, Address = 0x7ffd42a1b23c
Fb: Value = 4.5, Address = 0x7ffd42a1b238
Chvar: Value = Z, Address = 0x7ffd42a1b237
```

Question 2: Integer Variable Swap

Declare two integer variables, assign values, print addresses, swap contents, and check if addresses remain same.

Answer:

```
#include <stdio.h>

int main() {
    int a = 10, b = 20;

    printf("Before swap:\n");
```

```

printf("a: Value = %d, Address = %p\n", a, &a);
printf("b: Value = %d, Address = %p\n", b, &b);

int temp = a;
a = b;
b = temp;

printf("After swap:\n");
printf("a: Value = %d, Address = %p\n", a, &a);
printf("b: Value = %d, Address = %p\n", b, &b);

return 0;
}

```

Output:

Before swap:
a: Value = 10, Address = 0x7ffd42a1b234
b: Value = 20, Address = 0x7ffd42a1b230
After swap:
a: Value = 20, Address = 0x7ffd42a1b234
b: Value = 10, Address = 0x7ffd42a1b230
Addresses remain the same before and after swap.

Question 3: Pointer Declaration and Initialization

Declare and initialize pointer variable p to display values of x, y, and z.

Answer:

```

#include <stdio.h>

int main() {
    float x = 6.7, y = 1.2, z = 2.3;
    float *p;

    p = &x;
    printf("x = %.1f\n", *p);

    p = &y;
    printf("y = %.1f\n", *p);

    p = &z;
    printf("z = %.1f\n", *p);

    return 0;
}

```

Output:

```
x = 6.7
y = 1.2
z = 2.3
```

Question 4: Multiple Pointers to Same Variable

Declare and initialize pointer variables p1, p2, p3 for variable x, display value from p1, and update value using p3.

Answer:

```
#include <stdio.h>

int main() {
    int x = 89;
    int *p1, *p2, *p3;

    p1 = &x;
    p2 = &x;
    p3 = &x;

    printf("Value of x from p1: %d\n", *p1);
    *p3 = 100;
    printf("Updated value of x: %d\n", x);

    return 0;
}
```

Output:

```
Value of x from p1: 89
Updated value of x: 100
```

Question 5: Updating Variables Through Single Pointer

Update values of a, b, and c to be incremented by 10 through a single pointer variable.

Answer:

```
#include <stdio.h>

int main() {
    int a = 12, b = 25, c = 18;
```

```

int *ptr;

printf("Before increment:\n");
printf("a = %d, b = %d, c = %d\n", a, b, c);

ptr = &a;
*ptr += 10;

ptr = &b;
*ptr += 10;

ptr = &c;
*ptr += 10;

printf("After increment:\n");
printf("a = %d, b = %d, c = %d\n", a, b, c);

return 0;
}

```

Output:

Before increment:
 a = 12, b = 25, c = 18
 After increment:
 a = 22, b = 35, c = 28

Question 6: Multiple Pointers for Multiple Variables

Update values of a, b, and c using their respective pointers.

Answer:

```

#include <stdio.h>

int main() {
    int a = 12, b = 52, c = 8;
    int *ptr1, *ptr2, *ptr3;

    ptr1 = &a;
    ptr2 = &b;
    ptr3 = &c;

    printf("Before increment:\n");
    printf("a = %d, b = %d, c = %d\n", a, b, c);

    *ptr1 += 10;
    *ptr2 += 10;
    *ptr3 += 10;

    printf("After increment:\n");
    printf("a = %d, b = %d, c = %d\n", a, b, c);
}

```

```
    return 0;
}
```

Output:

Before increment:

a = 12, b = 52, c = 8

After increment:

a = 22, b = 62, c = 18

Question 7: Finding Greater Value Using Pointers

Find the greater between a and b using pointer manipulation.

Answer:

```
#include <stdio.h>

int main() {
    int a = 52, b = 18;
    int *ptr1 = &a, *ptr2 = &b;

    if (*ptr1 > *ptr2) {
        printf("%d is greater than %d\n", *ptr1, *ptr2);
    } else {
        printf("%d is greater than %d\n", *ptr2, *ptr1);
    }

    return 0;
}
```

Output:

52 is greater than 18

Question 8: Array Elements Address and Value

Display address and value of each element of integer array a.

Answer:

```
#include <stdio.h>

int main() {
    int a[10] = {0, 10, 20, 30, 40, 50, 60, 70, 80, 90};

    printf("Index\tValue\tAddress\n");
    for(int i = 0; i < 10; i++) {
        printf("%d\t%d\t%p\n", i, a[i], &a[i]);
    }

    return 0;
}
```

Output:

Index	Value	Address
0	0	0x7ffd42a1b240
1	10	0x7ffd42a1b244
2	20	0x7ffd42a1b248
3	30	0x7ffd42a1b24c
4	40	0x7ffd42a1b250
5	50	0x7ffd42a1b254
6	60	0x7ffd42a1b258
7	70	0x7ffd42a1b25c
8	80	0x7ffd42a1b260
9	90	0x7ffd42a1b264

Question 9: Array Values Through Pointers

Print values and addresses of integer and float arrays through pointers.

Answer:

```
#include <stdio.h>

int main() {
    int int_arr[5] = {10, 13, 20, 33, 44};
    float float_arr[6] = {10.2, 13.3, 20.0, 33.3, 45.3, 89.9};

    printf("Integer array:\n");
    for(int i = 0; i < 5; i++) {
        printf("Value: %d, Address: %p\n", *(int_arr + i), int_arr + i);
    }

    printf("Float array:\n");
    for(int i = 0; i < 6; i++) {
        printf("Value: %.1f, Address: %p\n", *(float_arr + i), float_arr + i);
    }

    return 0;
}
```

Output:

Integer array:
Value: 10, Address: 0x7ffd42a1b270
Value: 13, Address: 0x7ffd42a1b274
Value: 20, Address: 0x7ffd42a1b278
Value: 33, Address: 0x7ffd42a1b27c
Value: 44, Address: 0x7ffd42a1b280
Float array:
Value: 10.2, Address: 0x7ffd42a1b288
Value: 13.3, Address: 0x7ffd42a1b28c
Value: 20.0, Address: 0x7ffd42a1b290
Value: 33.3, Address: 0x7ffd42a1b294
Value: 45.3, Address: 0x7ffd42a1b298
Value: 89.9, Address: 0x7ffd42a1b29c

Question 10: Multiple Pointers to Array Elements

Declare and initialize pointer variables for array elements and display array content.

Answer:

```
#include <stdio.h>

int main() {
    int a[6] = {120, 502, 118, 188, 106, 447};
    int *ptr1, *ptr2, *ptr3, *ptr4, *ptr5, *ptr6;

    ptr1 = &a[0];
    ptr2 = &a[1];
    ptr3 = &a[2];
    ptr4 = &a[3];
    ptr5 = &a[4];
    ptr6 = &a[5];

    printf("Array elements using multiple pointers:\n");
    printf("%d %d %d %d %d %d\n", *ptr1, *ptr2, *ptr3, *ptr4, *ptr5, *ptr6);

    return 0;
}
```

Output:

Array elements using multiple pointers:
120 502 118 188 106 447

Question 11: Single Pointer to Array

Display array content using single pointer.

Answer:

```
#include <stdio.h>

int main() {
    int a[6] = {120, 502, 118, 188, 106, 447};
    int *ptr = &a[0];

    printf("Array elements using single pointer:\n");
    for(int i = 0; i < 6; i++) {
        printf("%d ", *(ptr + i));
    }
    printf("\n");

    return 0;
}
```

Output:

Array elements using single pointer:
120 502 118 188 106 447

Question 12: Array Name as Pointer

Use array name as pointer to display array content.

Answer:

```
#include <stdio.h>

int main() {
    int a[6] = {120, 502, 118, 188, 106, 447};
    int *ptr = a;

    printf("Array elements using array name as pointer:\n");
    for(int i = 0; i < 6; i++) {
        printf("%d ", *(ptr + i));
    }
    printf("\n");
}
```



```
    return 0;
}
```

Output:

Array elements using array name as pointer:
120 502 118 188 106 447

Question 13: Code Fragment Tracing

Trace the execution of the given code fragment.

Answer:

```
#include <stdio.h>

int main() {
    int m = 10, n = 5;
    int *mp, *np;
    mp = &m;
    np = &n;
    *mp = *mp + *np;
    *np = *mp - *np;
    printf("%d %d\n%d %d\n", m, *mp, n, *np);

    return 0;
}
```

Output:

15 15
10 10

Question 14: Error Identification

Describe errors in each statement.

Answer:

```
#include <stdio.h>

int main() {
    int m = 25, n = 77;
    char c = '*';
    int *itemp;

    // m = &n;          // ERROR: Cannot assign 'int *' to 'int'
    // itemp = m;        // ERROR: Cannot assign 'int' to 'int *'
    // *itemp = c;       // ERROR: itemp uninitialized + char to int
    // *itemp = &c;      // ERROR: Cannot assign 'char *' to 'int'

    printf("All statements contain errors as described above.\n");
    return 0;
}
```

Output:

All statements contain errors as described above.

Question 15: Store Values and Find Maximum

Store values in variables using pointers and find maximum.

Answer:

```
#include <stdio.h>

int main() {
    int a, b, c, d, maxvar;
    int *ptr_a = &a, *ptr_b = &b, *ptr_c = &c, *ptr_d = &d, *ptr_max = &maxvar;

    *ptr_a = 55;
    *ptr_b = 105;
    *ptr_c = 89;
    *ptr_d = 68;

    *ptr_max = *ptr_a;
    if (*ptr_b > *ptr_max) *ptr_max = *ptr_b;
    if (*ptr_c > *ptr_max) *ptr_max = *ptr_c;
    if (*ptr_d > *ptr_max) *ptr_max = *ptr_d;

    printf("Values: a=%d, b=%d, c=%d, d=%d\n", a, b, c, d);
    printf("Maximum: %d\n", maxvar);

    return 0;
}
```

Output:

Values: a=55, b=105, c=89, d=68
Maximum: 105

Question 16: Element-wise Array Sum

Find element-wise sum of arrays using pointers.

Answer:

```
#include <stdio.h>

int main() {
    int a[5], b[5], c[5], d[5], sumarr[5];
    int *ptr_a = a, *ptr_b = b, *ptr_c = c, *ptr_d = d, *ptr_sum = sumarr;

    printf("Enter 5 elements for array a: ");
    for(int i = 0; i < 5; i++) scanf("%d", ptr_a + i);

    printf("Enter 5 elements for array b: ");
    for(int i = 0; i < 5; i++) scanf("%d", ptr_b + i);

    printf("Enter 5 elements for array c: ");
    for(int i = 0; i < 5; i++) scanf("%d", ptr_c + i);

    printf("Enter 5 elements for array d: ");
    for(int i = 0; i < 5; i++) scanf("%d", ptr_d + i);

    for(int i = 0; i < 5; i++) {
        *(ptr_sum + i) = *(ptr_a + i) + *(ptr_b + i) + *(ptr_c + i) + *(ptr_d + i);
    }

    printf("Element-wise sum: ");
    for(int i = 0; i < 5; i++) printf("%d ", *(ptr_sum + i));
    printf("\n");

    return 0;
}
```

Output:

```
Enter 5 elements for array a: 1 2 3 4 5
Enter 5 elements for array b: 2 3 4 5 6
Enter 5 elements for array c: 3 4 5 6 7
Enter 5 elements for array d: 4 5 6 7 8
Element-wise sum: 10 14 18 22 26
```

Question 17: String Array Manipulation

Manipulate character array to hold uppercase letters using pointers.

Answer:

```
#include <stdio.h>
#include <ctype.h>

int main() {
    char *array[] = {"mine", "cs", "sc", "soa", NULL};
    char *ptr;

    printf("Original strings:\n");
    for(int i = 0; array[i] != NULL; i++) {
        printf("%s\n", array[i]);
    }

    printf("Uppercase strings:\n");
    for(int i = 0; array[i] != NULL; i++) {
        ptr = array[i];
        while(*ptr) {
            printf("%c", toupper(*ptr));
            ptr++;
        }
        printf("\n");
    }

    return 0;
}
```

Output:

```
Original strings:
mine
cs
sc
soa
Uppercase strings:
MINE
CS
SC
SOA
```

Question 18: Memory Allocation for Nameless Variables

State method to allocate memory for pointers to manipulate desired variables.

Answer:

For nameless variables, arrays, and strings:

1. Use dynamic memory allocation with malloc()
2. For arrays: `int *arr = (int*)malloc(size * sizeof(int));`
3. For strings: `char *str = (char*)malloc(size * sizeof(char));`
4. For string arrays: `char **arr = (char**)malloc(rows * sizeof(char*));`
5. Remember to free allocated memory using free()

Output:

Memory allocation methods described above.

Question 19-20: Function sum_n_avg

Write prototype and implementation for sum_n_avg function.

Answer:

```
#include <stdio.h>

void sum_n_avg(double n1, double n2, double n3, double *sum, double *avg);

int main() {
    double one, two, three, sum_of_3, avg_of_3;
    printf("Enter three numbers> ");
    scanf("%lf%lf%lf", &one, &two, &three);
    sum_n_avg(one, two, three, &sum_of_3, &avg_of_3);
    printf("Sum: %.2f, Average: %.2f\n", sum_of_3, avg_of_3);
    return 0;
}

void sum_n_avg(double n1, double n2, double n3, double *sum, double *avg) {
    *sum = n1 + n2 + n3;
    *avg = *sum / 3.0;
}
```

Output:

Enter three numbers> 10 20 30
Sum: 60.00, Average: 20.00

Question 21: Sorting with arrange() Function

Sort 6 numbers using arrange() function with input/output parameters.

Answer:

```
#include <stdio.h>

void arrange(int *a, int *b);

int main() {
    int n1, n2, n3, n4, n5, n6;
    printf("Enter SIX numbers separated by blanks> ");
    scanf("%d%d%d%d%d%d", &n1, &n2, &n3, &n4, &n5, &n6);

    arrange(&n1, &n2); arrange(&n1, &n3); arrange(&n1, &n4);
    arrange(&n1, &n5); arrange(&n1, &n6); arrange(&n2, &n3);
    arrange(&n2, &n4); arrange(&n2, &n5); arrange(&n2, &n6);
    arrange(&n3, &n4); arrange(&n3, &n5); arrange(&n3, &n6);
    arrange(&n4, &n5); arrange(&n4, &n6); arrange(&n5, &n6);

    printf("The numbers in ascending order are: %d %d %d %d %d %d\n",
           n1, n2, n3, n4, n5, n6);
    return 0;
}

void arrange(int *a, int *b) {
    if (*a > *b) {
        int temp = *a;
        *a = *b;
        *b = temp;
    }
}
```

Output:

```
Enter SIX numbers separated by blanks> 12 3 56 8 20 654
The numbers in ascending order are: 3 8 12 20 56 654
```

Question 22: Table of Values

Show table of values for x, y, and z from the given program.

Answer:

```
#include <stdio.h>

void sum(int a, int b, int *cp);

int main() {
```

```

int x, y, z;
x = 7; y = 2;
printf("x y z\n\n");
sum(x, y, &z);
printf("%4d%4d%4d\n", x, y, z);
sum(y, x, &z);
printf("%4d%4d%4d\n", x, y, z);
sum(z, y, &x);
printf("%4d%4d%4d\n", x, y, z);
sum(z, z, &x);
printf("%4d%4d%4d\n", x, y, z);
sum(y, y, &y);
printf("%4d%4d%4d\n", x, y, z);
return 0;
}

void sum(int a, int b, int *cp) {
    *cp = a + b;
}

```

Output:

```

x y z
  7  2  9
  7  2  9
 11  2  9
 18  2  9
 18  4  9

```

Question 23: Parameter Classification and Values

Find values and classify parameters as input/output.

Answer:

```

#include <stdio.h>

void double_trouble(int *p, int y);
void trouble(int *x, int *y);

int main() {
    int x, y;
    trouble(&x, &y);
    printf("x = %d, y = %d\n", x, y);
    return 0;
}

void double_trouble(int *p, int y) {
    int x = 10;
    *p = 2 * x - y;
}

```

```
void trouble(int *x, int *y) {
    double trouble(x, 7);
    double_trouble(y, *x);
}

// Parameter classification:
// double trouble: *p (output), v (input)
// trouble: *x (output), *y (output)
```

Output:

x = 13, y = 7

Question 24: String Reversal

Reverse a string using pointer manipulation.

Answer:

```
#include <stdio.h>
#include <string.h>

int main() {
    char str[100], temp;
    char *start, *end;

    printf("Enter a string: ");
    fgets(str, sizeof(str), stdin);
    str[strcspn(str, "\n")] = 0;

    start = str;
    end = str + strlen(str) - 1;

    while(start < end) {
        temp = *start;
        *start = *end;
        *end = temp;
        start++;
        end--;
    }

    printf("Reversed string: %s\n", str);
    return 0;
}
```

Output:

Enter a string: hello
Reversed string: olleh

Question 25: Largest Value in Array

Find largest value in 1-D array using pointers.

Answer:

```
#include <stdio.h>

int main() {
    int arr[100], n, largest;
    int *ptr = arr;

    printf("Enter number of elements: ");
    scanf("%d", &n);

    printf("Enter %d elements: ", n);
    for(int i = 0; i < n; i++) {
        scanf("%d", ptr + i);
    }

    largest = *ptr;
    for(int i = 1; i < n; i++) {
        if(*(ptr + i) > largest) {
            largest = *(ptr + i);
        }
    }

    printf("Largest value: %d\n", largest);
    return 0;
}
```

Output:

Enter number of elements: 5
Enter 5 elements: 23 45 12 67 34
Largest value: 67

Question 26: Matrix Transpose

Transpose of a matrix using pointers.

Answer:

```
#include <stdio.h>

int main() {
    int matrix[10][10], transpose[10][10];
    int rows, cols;

    printf("Enter rows and columns: ");
    scanf("%d%d", &rows, &cols);

    printf("Enter matrix elements:\n");
    for(int i = 0; i < rows; i++) {
        for(int j = 0; j < cols; j++) {
            scanf("%d", (*(matrix + i) + j));
        }
    }

    for(int i = 0; i < rows; i++) {
        for(int j = 0; j < cols; j++) {
            (*(transpose + j) + i) = (*(matrix + i) + j);
        }
    }

    printf("Transpose matrix:\n");
    for(int i = 0; i < cols; i++) {
        for(int j = 0; j < rows; j++) {
            printf("%d ", (*(transpose + i) + j));
        }
        printf("\n");
    }

    return 0;
}
```

Output:

```
Enter rows and columns: 2 3
Enter matrix elements:
1 2 3
4 5 6
Transpose matrix:
1 4
2 5
3 6
```